

Task 1.1

Relation A.

1. {empID, Name}, {empID, Department}, {SSN, Name}, {empID, phone}, {salary, empID}, {SSN, phone}
2. {empID}, {SSN}
3. EmpID is the best option to be chosen for primary key because of the fact that not all employees have social security number. They may be foreign workers that do not have this unique number which can cause null value, also SSN is sensitive information
4. two people cannot have the same phone number but they can change it

Relation B.

1. {StudentID, CourseCode, Semester, Year}
2. StudentID to determine certain student that is taking course. Course code is also required to be known because of this. And to know particular time of taking up this course year and semester are need to be the atributes of primary key.
3. If a unique SectionID exists that inherently includes the course code, then {StudentID, SectionID, Semester, Year} could serve as an alternative candidate key.

Task 1.2

1. Student(AdvisorID) references Professor(ProfID)
Department(ChairID) references Professor(ProfID)
Enrollment(StudentID) references Student(StudentID)
Enrollment(CourseID) references Course(CourseID)

Task 2.1

1. Strong entities: Patients, Doctors, Departments.

Weak entities: Appointments, Prescriptions, Hospital Rooms.

2. Patient: PatientID (simple), Name (simple), Birthdate (simple), Address (composite: street, city, state, zip), Phone (multi-valued), InsuranceInfo (simple).

Doctor: DoctorID (simple), Name (simple), Specialization (multi-valued), Phone (simple), OfficeLocation (simple).

Department: DeptCode (simple), DeptName (simple), Location (simple).

Room: RoomNumber.

Appointment: AppointmentDate (simple), AppointmentTime (simple), Purpose (simple), Notes (simple).

Prescription: MedicationName (simple), Dosage (simple), Instructions (simple).

3. Customer - Order: 1:N.

Category - Product: 1:N.

Vendor - Product: 1:N.

Order - Product : M:N.

4, 5.

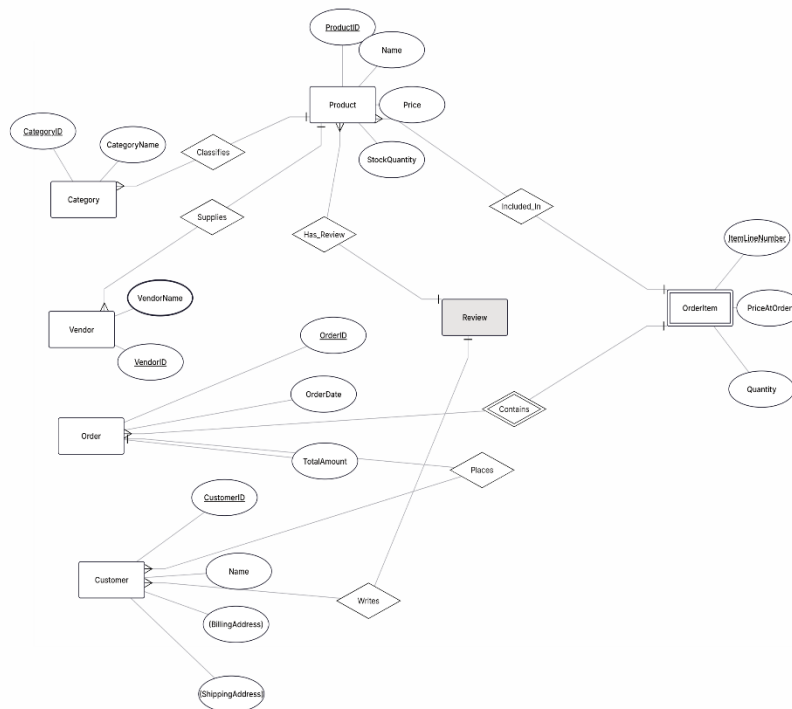
The ER diagram illustrates the following entities and their attributes:

- Patient** (Entity): Phone, PatientID (PK), City, Street, State, Zip, Name, Birthdate.
- Doctor** (Entity): DoctorID (PK), Specialization (PK), Name, Phone, OfficeLocation.
- Department** (Entity): DeptCode (PK), DeptName, Location.
- Appointment** (Entity): AppointmentDate, AppointmentTime, Purpose, Notes.
- Prescription** (Entity): Dosage, Instructions, MedicationName.
- Room** (Entity): RoomNumber (PK).

The relationships and their cardinalities are:

- Schedules** (1:M): Connects **Patient** (1) and **Appointment** (M).
- WorksIn** (1:M): Connects **Doctor** (1) and **Department** (M).
- HasRoom** (1:M): Connects **Department** (1) and **Room** (M).
- Prescribes** (1:M): Connects **Doctor** (1) and **Prescription** (M).
- Prescribes** (1:M): Connects **Room** (1) and **Prescription** (M).

3. A many-to-many relationship between Order and Product is resolved using OrderItem as an associative entity. It includes two key attributes: Quantity, which tracks the number of units purchased, and PriceAtOrder, which records the product's unit price at the time of purchase to preserve historical financial accuracy despite future price changes.



Task 4.1

1. Functional Dependencies (FDs):

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

2. Redundancy & Anomalies:

- **Redundancy:** StudentName and StudentMajor are repeated for every project a student joins. ProjectTitle, ProjectType, SupervisorName, and

SupervisorDept are duplicated across all students assigned to the same project.

- **Insertion Anomaly:** You cannot add a new project into the database unless at least one student is assigned to it, because StudentID is part of the primary key.
- **Deletion Anomaly:** If the only student assigned to a specific project is deleted, all historical data regarding that project, its title, and its supervisor is permanently lost.
- **Update Anomaly:** If a supervisor changes their department (SupervisorDept), every single record containing that supervisor must be updated. Missing even one row leads to data inconsistency.

3. Apply 1NF:

- **Violations Check:** The table is already in 1NF assuming all attributes contain atomic (indivisible) values and there are no repeating groups.
- **Fix:** If fields like Role or Supervisor contained multiple comma-separated values, they would need to be split into individual rows with atomic attributes.

4. Apply 2NF:

- **Primary Key:** (StudentID, ProjectID)
- **Partial Dependencies:**
 - StudentID → StudentName, StudentMajor (depends only on part of the PK).
 - ProjectID → ProjectTitle, ProjectType, SupervisorID (depends only on part of the PK).
- **2NF Decomposition:**
 - Student (**StudentID**, StudentName, StudentMajor)
 - Project (**ProjectID**, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)
 - StudentProjectAssignment (**StudentID**, **ProjectID**, Role, HoursWorked, StartDate, EndDate)

5. Apply 3NF:

- **Transitive Dependencies:**

- In the Project table: $\text{ProjectID} \rightarrow \text{SupervisorID}$ and $\text{SupervisorID} \rightarrow \text{SupervisorName}, \text{SupervisorDept}$.

- **Final 3NF Decomposition:**

- Student (**StudentID**, StudentName, StudentMajor)
- Supervisor (**SupervisorID**, SupervisorName, SupervisorDept)
- Project (**ProjectID**, ProjectTitle, ProjectType, SupervisorID*)
- StudentProjectAssignment (**StudentID**, **ProjectID**, Role, HoursWorked, StartDate, EndDate)

Task 4.2

1. Primary Key Determination:

- The Primary Key is (**StudentID**, **CourseID**).
- Alternatively, (**StudentID**, **TimeSlot**, **Room**) can also serve as a candidate key since a course section occupies a specific time slot and room.

2. Functional Dependencies:

- $\text{StudentID} \rightarrow \text{StudentMajor}$
- $\text{CourseID} \rightarrow \text{CourseName}$
- $\text{InstructorID} \rightarrow \text{InstructorName}$
- $\text{Room} \rightarrow \text{Building}$
- $\text{CourseID} \rightarrow \text{InstructorID}, \text{TimeSlot}, \text{Room}$ (under the rule that a course section is taught by one instructor at one time in one room)
- $(\text{TimeSlot}, \text{Room}) \rightarrow \text{CourseID}$

3. BCNF Check:

- **Is it in BCNF? No.**
- A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey. Here, non-superkey determinants like StudentID, CourseID, InstructorID, and Room determine other attributes, violating BCNF.

4. BCNF Decomposition Steps:

1. **Remove Student info:** Student (**StudentID**, StudentMajor)
2. **Remove Course info:** Course (**CourseID**, CourseName)
3. **Remove Instructor info:** Instructor (**InstructorID**, InstructorName)
4. **Remove Room info:** RoomLocation (**Room**, Building)
5. **Create Section Schedule:** CourseSection (**CourseID**, InstructorID, *TimeSlot*, *Room*)
6. **Create Enrollment table:** StudentEnrollment (**StudentID**, *CourseID*)

5. Information Loss & Dependency Preservation Analysis:

- **No Loss of Data (Lossless Join):** The decomposition is lossless because all tables are joined via shared common keys (StudentID, CourseID, InstructorID, Room).
- **Dependency Preservation Note:** The functional dependency (TimeSlot, Room) → CourseID spans across CourseSection and StudentEnrollment. While BCNF decomposition successfully removes all anomalies, enforcing this cross-table constraint in a relational database may require database triggers or assertion checks to prevent two different courses from being scheduled in the exact same room at the same time.

Task 5.1

3 To model Club Officer Positions, a ternary associative entity named ClubOfficer (connecting Student, Club, and Position with StartDate and EndDate) was chosen over simply adding a role attribute to Membership. This decision enables historical tracking of leadership roles, allows a student to hold different officer positions across different academic years, and prevents update anomalies during officer transitions.

4. "Find all students who currently serve as officers in the Computer Science Club alongside their specific position titles.

"List all upcoming events scheduled for next week along with their assigned room numbers, building locations, and host clubs."

"Calculate the total approved expense amount for each club for the current fiscal year and compare it against their allocated total budget."

2. Student (**StudentID**, Name, Email, Major)

Faculty (**FacultyID**, Name, Email, Department)

Club (**ClubID**, ClubName, Category, FoundedDate, FacultyID*)

Membership (**StudentID***, **ClubID***, JoinDate, Status)

Position (**PositionID**, PositionTitle)

ClubOfficer (**StudentID***, **ClubID***, **PositionID***, StartDate, EndDate)

Room (**RoomID**, Building, RoomNumber, Capacity)

Event (**EventID**, EventTitle, EventDate, StartTime, EndTime, ClubID*, RoomID*)

Attendance (**StudentID***, **EventID***, CheckInTime)

Budget (**BudgetID**, TotalAmount, FiscalYear, ClubID*)

Expense (**ExpenseID**, Amount, Description, ExpenseDate, Status, BudgetID*)

1.

